

5. ALGORITHMS

This chapter presents the three core algorithms used in StegnoVault: AES-256 Encryption for data confidentiality, the Least Significant Bit (LSB) algorithm for data hiding across three media types, and the Genetic Algorithm (GA) for optimal embedding position selection. These algorithms work together in a three-layer pipeline to ensure that secret messages are both unreadable and undetectable.

5.1 Algorithm 1: AES-256 Encryption Algorithm

Algorithm Purpose

The AES-256 encryption algorithm ensures that the secret message remains confidential throughout the transmission process. Before any steganographic embedding takes place, AES transforms the plaintext into ciphertext through multiple rounds of substitution, permutation, and key transformation operations. This means that even if an attacker performs steganalysis and successfully extracts the hidden bits from the stego-file, the recovered data is computationally indistinguishable from random data without the correct 256-bit AES key. The key space of 2^{256} makes brute-force attacks computationally infeasible. AES is implemented in StegnoVault using the AES/CBC/PKCS5Padding mode via Java Cryptography Extension (JCE) with Unlimited Strength policy for 256-bit key support.

Input

- Secret plaintext message to be hidden (entered by the user in the Encoding Hub).
- 256-bit AES encryption key (derived from the user-provided key string).
- Randomly generated Initialization Vector (IV) for CBC mode.

Detailed Algorithm Steps — Encryption

1. **Key Expansion:** The 256-bit secret key is expanded into 15 round keys using the Rijndael key schedule algorithm (14 encryption rounds for AES-256).
2. **Initial Round — AddRoundKey:** Each byte of the first plaintext block is XORed with the corresponding byte of the first round key to introduce confusion.
3. **Main Rounds (Rounds 1 to 13):** Each round applies four transformations in sequence:
 - **SubBytes:** Each byte of the 4×4 state matrix is substituted with the corresponding value from the AES S-box (a fixed lookup table), introducing non-linearity.
 - **ShiftRows:** Rows of the 4×4 state matrix are cyclically shifted by 0, 1, 2, and 3 positions respectively, increasing diffusion.
 - **MixColumns:** Each column of the state matrix is multiplied by a fixed polynomial in $GF(2^8)$, creating interdependency between bytes within each column.
 - **AddRoundKey:** The state is XORed with the current round key, produced by the key schedule.
4. **Final Round (Round 14):** Applies SubBytes, ShiftRows, and AddRoundKey only. MixColumns is omitted in the final round.
5. **Ciphertext Output:** The result of the 14-round transformation is the AES-256 ciphertext. It is then converted to a binary bit string for LSB embedding.

Detailed Algorithm Steps — Decryption

1. Input: Extracted ciphertext binary string from the stego-file; same 256-bit AES key and IV used during encryption.
2. Inverse operations are applied in reverse order across 14 rounds: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
3. Final Output: Original plaintext message is recovered exactly.

Output

- Encryption Output: Binary ciphertext bit string ready for LSB embedding.
- Decryption Output: Original plaintext secret message recovered after extraction.

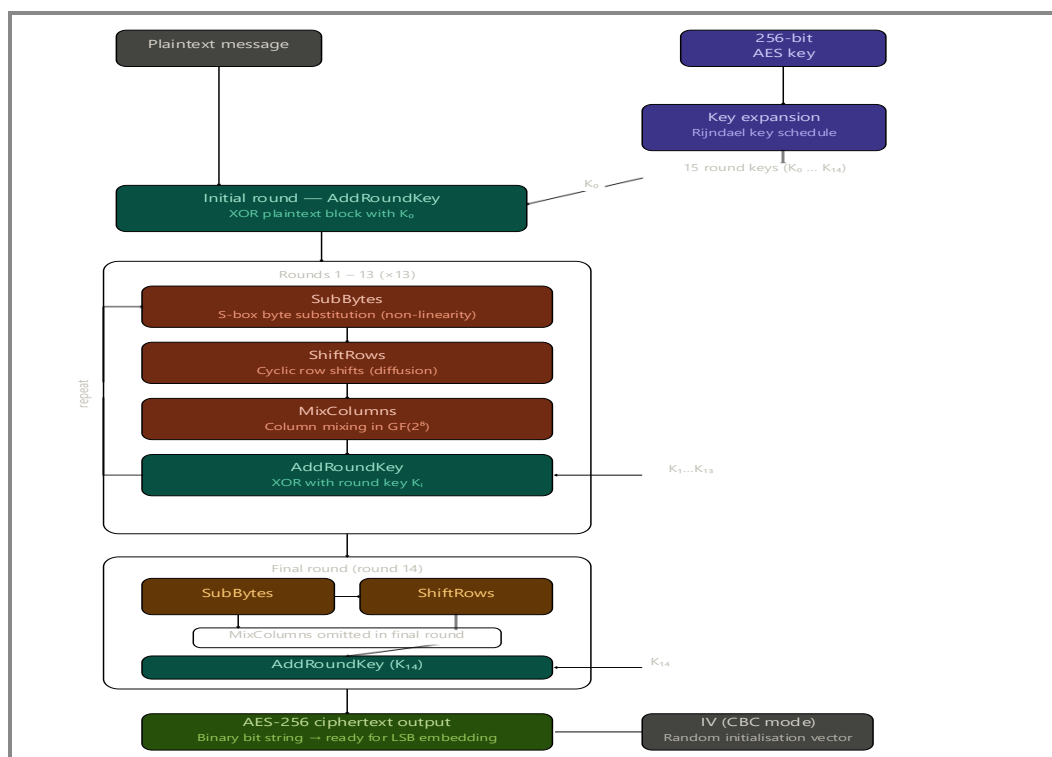


Fig. 5.1: AES-256 Algorithm Flow

5.2 Algorithm 2: LSB (Least Significant Bit) Steganography Algorithm

Algorithm Purpose

The LSB algorithm is one of the most efficient and widely used steganographic techniques for concealing information within digital media without visibly or audibly altering the original content. In StegnoVault, the LSB algorithm is applied after AES-256 encryption of the secret message, ensuring both confidentiality and invisibility. The technique modifies only the least significant bit of selected pixels (for images) or sample bytes (for audio and video), making changes imperceptible to human senses. Since the LSB represents the smallest possible change in a pixel or sample value (± 1 out of 256 for 8-bit channels), the visual and auditory quality of the stego-file is maintained.

5.2.1 Image LSB Steganography (ImageSteganographyService)

Steganographic embedding in digital images modifies pixel data to carry the secret binary payload.

Input: Cover image file (PNG or JPEG), AES ciphertext bit string, GA-selected pixel position array (int[]).

Detailed Steps — Encoding:

1. The cover image is loaded and converted to a BufferedImage of type TYPE_INT_RGB, normalizing all pixel formats including JPEG source files.
2. The AES ciphertext binary bit string is prepared with a #END delimiter appended at the end (e.g., AES ciphertext bits + #END marker bits).
3. The GA-generated position sequence (int[]) is retrieved. This sequence specifies the order in which pixels will be modified.
4. For each bit in the ciphertext bit string:
 - Retrieve the pixel at the GA-selected position from the int[] array.
 - Extract the blue channel value of the pixel (using bit masking: blueValue = pixel & 0xFF).
 - Clear the LSB of the blue channel: blueValue = blueValue & 0xFE.
 - Set the LSB to the current ciphertext bit: blueValue = blueValue | currentBit.
 - Reconstruct the full pixel with the modified blue channel and write it back.
5. Repeat step 4 for all bits in the ciphertext + delimiter.
6. Output the modified image as a lossless PNG file to prevent any compression-induced LSB corruption.

LSB Embedding Example (Image):

- Original pixel value: 10101100 (decimal 172, blue channel)
- Secret bit to embed: 1
- Modified pixel value: 10101101 (decimal 173)
- Change in pixel intensity: 1 out of 256 — completely imperceptible to human vision.

Detailed Steps — Decoding:

1. Load the stego-PNG file.
2. Reproduce the GA position sequence using the same GA seed (GeneticOptimizerService.generatePositions(gaSeed, imageSize)).
3. For each position in the sequence, read the pixel's blue channel and extract its LSB.
4. Accumulate bits into bytes. After every 8 bits, form a character and add to the accumulated string.
5. Check for the #END delimiter. When detected, stop extraction.
6. The accumulated bytes before the delimiter form the AES ciphertext.
7. Call AESCipherService.decrypt(ciphertext, aesKey) to recover the original plaintext.

5.2.2 Audio LSB Steganography (AudioSteganographyService)

Audio steganography hides data in the raw byte stream of WAV audio files without causing audible distortion.

Input: WAV audio file (raw byte array), AES ciphertext bit string, GA-selected byte position array (int[]).

Detailed Steps — Encoding:

1. Read the WAV file as a raw byte array using Java's file I/O.
2. Prepare the AES ciphertext bit string with the `##AUDIO##` binary marker appended.
3. Retrieve the GA-generated position sequence for the audio byte array.
4. For each bit in the ciphertext + marker: read the selected audio sample byte; clear its LSB; set it to the current ciphertext bit; write the modified byte back.
5. Output the modified byte array as a new WAV file. The WAV container format is preserved.

Detailed Steps — Decoding:

1. Read the stego-WAV file as a raw byte array.
2. Reproduce the GA position sequence. Extract LSBs from GA-selected positions.
3. Accumulate bits into bytes until the `##AUDIO##` marker binary pattern is detected.
4. Call `AESCipherService.decrypt()` to recover plaintext.

5.2.3 Video LSB Steganography (VideoSteganographyService)

Video steganography hides data in the raw frame byte stream of MP4 files without affecting playback quality.

Input: MP4 video file (raw frame byte data), AES ciphertext bit string, GA-selected byte position array (int[]).

Detailed Steps — Encoding:

1. Read the MP4 file's raw frame byte data.
2. Prepare the AES ciphertext bit string with the `##VIDEO##` binary marker appended.
3. Retrieve the GA position sequence for the video byte array.
4. For each bit: read the selected frame byte; modify its LSB; write it back.
5. The MP4 container format tolerates modifications within frame data without breaking playback. Standard video players read only up to the declared stream boundary, confirming covert embedding.
6. Output the modified byte stream as a new MP4 file. Container integrity is verified using `ffprobe`.

Detailed Steps — Decoding:

1. Read the stego-MP4 raw byte data.

2. Reproduce GA position sequence. Extract LSBs from selected positions until ##VIDEO## marker detected.
3. Call AESCipherService.decrypt() to recover plaintext.

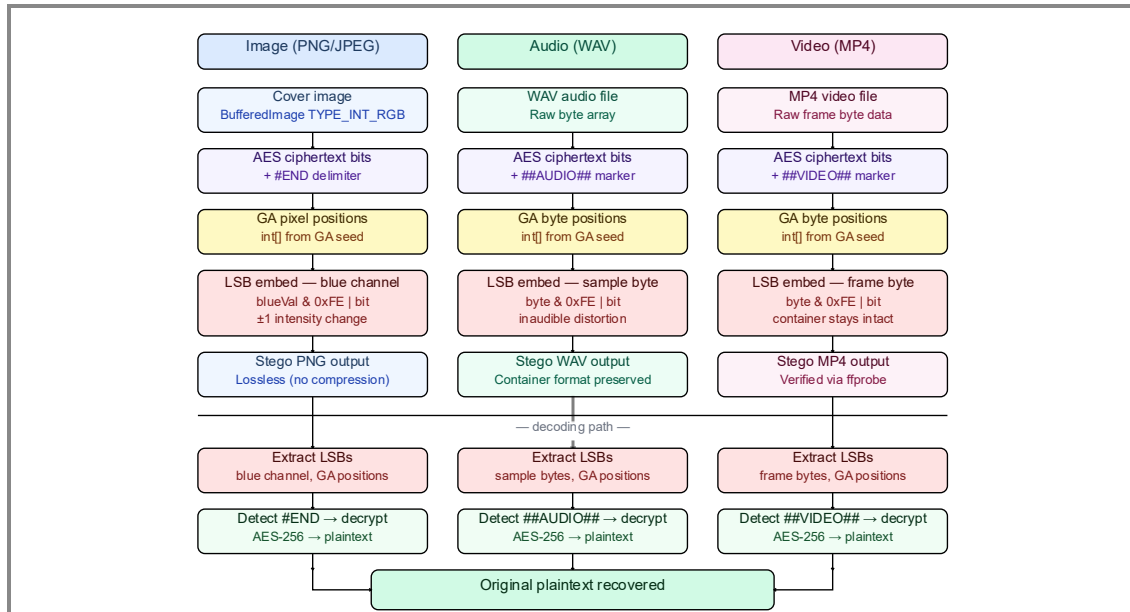


Fig. 5.2: LSB Algorithm Flow for Three Media Types

5.3 Algorithm 3: Genetic Algorithm (GA) Optimization

Algorithm Purpose

The Genetic Algorithm (GA) is implemented in GeneticOptimizerService to intelligently select the most suitable embedding positions within the cover media for LSB bit insertion. Sequential LSB traversal (embedding bits at positions 0, 1, 2, 3... in order) is well-known to be vulnerable to chi-squared and RS steganalysis attacks because it introduces detectable statistical patterns in the LSB plane. The GA overcomes this by selecting high-entropy, spatially distributed positions that are statistically inconspicuous. The same GA seed shared between the sender and receiver allows exact reproduction of the position sequence during decoding, without transmitting the map itself. The GA seed thus acts as a second key in addition to the AES key, requiring an attacker to know both keys for successful decoding.

GA Configuration Parameters

Parameter	Value
Population Size	50 candidate position sequences
Crossover Rate	0.8 (single-point crossover)
Mutation Rate	0.01 (random-bit mutation)
Max Generations	100

Selection Method	Tournament Selection
GA Seed	Shared between sender and receiver as extraction key
Fitness Criteria	Entropy Score + Spatial Distribution Score

Input

- GA seed (integer provided by the user — must be shared with the receiver).
- Media size (total number of pixels for images, total bytes for audio/video).

Detailed Algorithm Steps

1. Initial Population Generation: Using the provided GA seed as the random number generator seed for reproducibility, generate 50 candidate position sequences. Each sequence is an ordered list of N indices into the cover media byte/pixel array, where N is the number of bits to embed.
2. Fitness Function Evaluation: For each candidate sequence, compute the fitness score as a weighted combination of:
 - Entropy Score: Positions with higher local entropy (in images: pixels in textured regions; in audio: samples in acoustically complex regions) tolerate 1-bit modifications with minimal perceptual impact. Entropy is estimated using a local variance measure.
 - Spatial Distribution Score: Positions that are well-spread across the media are less detectable by block-based steganalysis tools that analyze statistical properties of 8×8 or 16×16 pixel/sample blocks. Measured as the standard deviation of position gaps.
3. Tournament Selection: From the population, randomly select 5 candidate sequences (tournament size). The sequence with the highest fitness score wins and is selected as a parent for the next generation.
4. Single-Point Crossover (rate 0.8): For each pair of selected parents, with probability 0.8, select a random crossover point. Exchange the position sub-sequences after the crossover point between the two parents to produce two offspring sequences.
5. Random-Bit Mutation (rate 0.01): For each bit in each offspring sequence, with probability 0.01, randomly replace the position index with a new random valid index. This maintains genetic diversity and prevents premature convergence.
6. Replacement: The offspring population replaces the parent population.
7. Convergence Check: Repeat steps 2–6 for up to 100 generations. If the best fitness score stabilizes (no improvement over 10 consecutive generations), stop early.
8. Output: The highest-fitness position sequence from the final generation is returned as the int[] embedding map. This map is used by the media-specific steganography service for LSB embedding.
9. Reproducibility: Since the GA is seeded with the user-provided GA seed, the same seed always produces the identical position sequence. The receiver uses the same seed to reproduce the map during decoding, enabling accurate extraction without transmitting the map.

Output

- `int[]` positions: An ordered array of position indices (pixel indices for images, byte indices for audio/video) at which the LSB bits will be embedded or extracted.

Performance Results

The GA optimization produced the following improvements in image steganography quality compared to sequential LSB embedding at equivalent payload sizes:

Method	Avg. PSNR (dB)	Avg. MSE	Avg. SSIM
GA-Optimized LSB	51.8 dB	< 0.005	> 0.9997
Sequential LSB	47.3 dB	> 0.010	0.9993
Improvement	+4.5 dB	−50% reduction	+0.0004

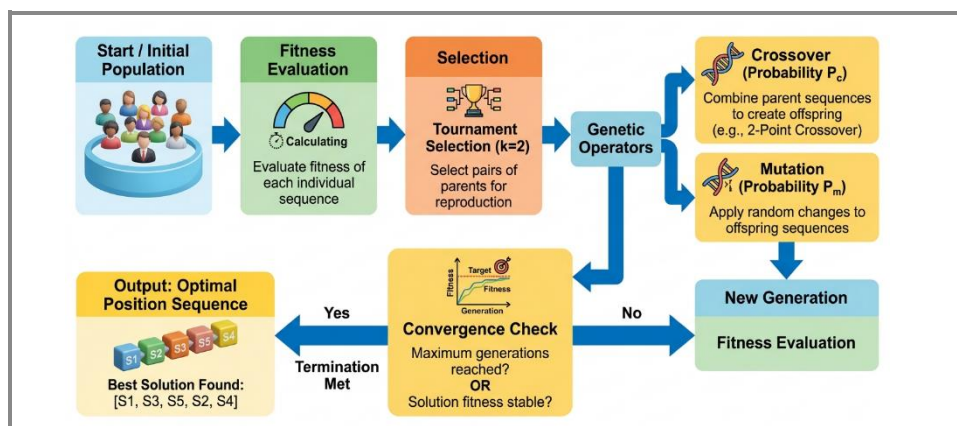


Fig. 5.3: Genetic Algorithm Flow Diagram

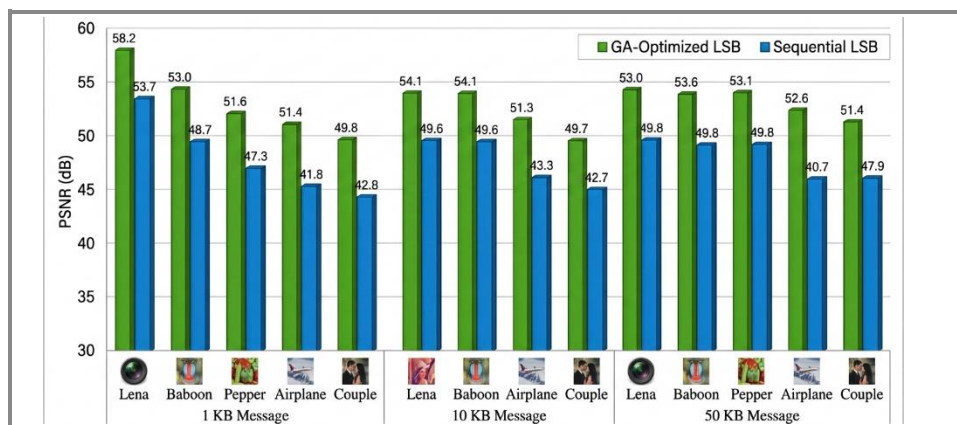


Fig. 5.4: PSNR Comparison — GA-Optimized LSB vs Sequential LSB

6. PROJECT IMPLEMENTATION

This chapter describes the complete implementation of StegnoVault in a phase-by-phase manner, documenting the development environment, each implementation phase with its activities and issues resolved, and the full working model of the proposed system. The project was developed over a 24-week implementation period from Week 17 (November 2025) to Week 40 (April 2026).

6.1 Project Steps

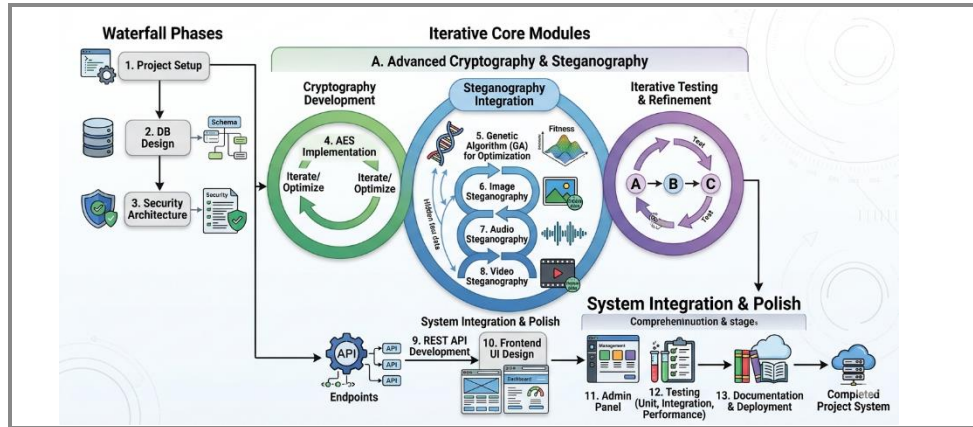


Fig. 6.1: Project Steps Diagram

Phase 1: Project Review and Technology Decision

Goal: Review the Stage I desktop application and decide on the architecture for the final web-based system.

Activities:

- Reviewed Stage I report covering LSB steganography implemented as a Java Swing desktop GUI application.
- Guide Prof. Yogita More suggested migrating the project to a fully dynamic web-based application for better accessibility, scalability, and real-world applicability.
- Decided to redesign the project as a web-based system using Java Spring Boot (backend), MySQL 8.0 (database), and HTML5/CSS3/JavaScript (frontend).
- Planned new system architecture: REST API backend, responsive web browser frontend, MySQL database.
- Prepared a revised project plan covering the web migration roadmap and updated technology stack.

Phase 2: Development Environment Setup

Goal: Configure all required development tools, frameworks, and dependencies.

Technology Stack Selected:

- Programming Language: Java 17
- Backend Framework: Spring Boot 3.x with Maven build tool
- Dependencies: Spring Web, Spring Security, Spring Data JPA, Spring Mail, MySQL Connector/J, Thymeleaf

- Database: MySQL 8.0
- IDE: IntelliJ IDEA
- Frontend: HTML5, CSS3, JavaScript ES6+, Google Fonts, Font Awesome 6.5

Activities:

- Created Spring Boot 3.x project with Maven using Spring Initializr.
- Added all required dependencies to pom.xml.
- Configured application.properties for MySQL 8.0 data source, JPA/Hibernate settings, and embedded Tomcat port.
- Verified application starts successfully on embedded Tomcat server with no errors.

Issue Encountered: Dependency version conflicts between Spring Boot 3.x and older Spring Security versions.

Corrective Measure: Upgraded all dependencies to Spring Boot 3.x compatible versions and resolved Maven dependency conflicts by cleaning the Maven local repository.

Phase 3: Database Schema Design and JPA Entity Implementation

Goal: Design and create the MySQL database schema; implement JPA entity classes.

Activities:

- Created MySQL 8.0 database with two tables:
 - users table: id (PK, auto-increment), username (VARCHAR, unique — email), name, mobile, password (BCrypt), role (ENUM), verified (BOOLEAN), active (BOOLEAN), bio (TEXT), profile_picture (VARCHAR), last_login (TIMESTAMP).
 - upload_history table: id (PK, auto-increment), username (VARCHAR, FK referencing users.username), media_type (ENUM: IMAGE/AUDIO/VIDEO), action_type (ENUM: ENCODE/DECODE), filename (VARCHAR), timestamp (TIMESTAMP).
- Implemented User JPA entity class with @Entity, @Table(name='users'), @Column annotations and all field mappings.
- Implemented UploadHistory JPA entity class with @Entity, @Table(name='upload_history').
- Configured UserRepository and UploadHistoryRepository extending JpaRepository with custom query methods.

Issue Encountered: Foreign key constraint errors during initial table creation due to incorrect entity relationship mappings.

Corrective Measure: Corrected entity relationship mappings and JPA cascade types to resolve constraint issues.

Phase 4: Security Implementation — BCrypt and Spring Security RBAC

Goal: Implement password hashing and role-based access control.

Activities:

- Configured BCrypt password encoder with strength 12 in PasswordConfig bean.

- Defined Spring Security filter chain in SecurityConfig with complete permit/restrict rules: public access to /register, /login, /verify-otp, /forgot-password, /reset-password; ROLE_USER access to /dashboard, /api/encode/**, /api/decode/**; ROLE_ADMIN access to /admin.html, /admin/**.
- Restricted /admin.html and all /admin/** endpoints exclusively to ROLE_ADMIN.
- Created custom 403 error page for unauthorized access attempts (displayed when ROLE_USER tries to access admin endpoints).
- Admin account created by directly inserting a record into MySQL with role=ROLE_ADMIN, BCrypt-hashed password, verified=true, active=true. This ensures admin credentials are never exposed through the public registration interface.

Issue Encountered: Spring Security filter chain ordering caused some endpoints to be unintentionally blocked (403 returned for public registration endpoint).

Corrective Measure: Corrected filter chain permit rules using .permitAll() for public endpoints and tested all endpoint access for both roles using Postman.

Phase 5: OTP-Based Email Verification

Goal: Implement OTP-based email verification for user registration.

Activities:

- Configured Spring Mail with Gmail SMTP in application.properties (host: smtp.gmail.com, port: 587, TLS enabled).
- Implemented OtpService: 6-digit OTP generated using SecureRandom (cryptographically secure), stored in the database with a 10-minute expiry timestamp.
- Built AuthController endpoints: POST /register (creates unverified user and triggers OTP), POST /verify-otp (validates OTP and sets verified=true), POST /resend-otp (generates new OTP and sends again).
- Tested OTP delivery to registered email addresses successfully.

Issue Encountered: Gmail SMTP authentication failed initially due to missing app-specific password (Gmail 2FA blocks regular password usage for SMTP).

Corrective Measure: Enabled 2-Factor Authentication on the Gmail account and generated an app-specific password for SMTP login in application.properties.

Phase 6: AES-256 Encryption Service

Goal: Implement the AES-256 encryption and decryption service.

Activities:

- Implemented AESCipherService using AES/CBC/PKCS5Padding mode with a 256-bit key via Java Cryptography Extension (JCE).
- AES performs SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations over 14 rounds for AES-256.
- Converted AES ciphertext output to binary bit string (a String of '0' and '1' characters) for LSB embedding.

- Verified decryption correctly recovers plaintext from ciphertext across all test cases (100, 500, 1000 character messages).

Issue Encountered: Initial key size configuration produced `InvalidKeyException: Illegal key size` — JDK does not allow 256-bit keys by default.

Corrective Measure: Installed Java Cryptography Extension (JCE) Unlimited Strength Jurisdiction Policy Files to enable AES-256. For Java 9+, this is enabled by default; for Java 8, the policy files must be installed.

Phase 7: Genetic Algorithm Optimizer Service

Goal: Implement the Genetic Algorithm for optimal embedding position selection.

Activities:

- Implemented `GeneticOptimizerService` with an initial population of 50 candidate position sequences.
- Fitness function evaluates each sequence on entropy score and spatial distribution score.
- Applied tournament selection, single-point crossover (rate 0.8), and random-bit mutation (rate 0.01).
- GA runs for a maximum of 100 generations; highest-fitness sequence selected as the embedding map.
- GA seed shared between sender and receiver as the extraction key; same seed always produces identical output (deterministic random using seeded `Random` instance).

Issue Encountered: GA convergence was slow for large cover images due to large population evaluation at every generation.

Corrective Measure: Tuned population size and selection pressure; reduced unnecessary fitness recalculations by caching entropy scores.

Phase 8: Image Steganography — Encode and Decode

Goal: Implement the LSB image steganography encoding and decoding service.

Encoding Activities:

- Implemented `ImageSteganographyService.encode()`: Converts cover image to `BufferedImage` `TYPE_INT_RGB`.
- AES ciphertext bit string with `#END` delimiter embedded one bit at a time into the LSB of the blue channel at GA-selected pixel positions.
- Stego-image always output as lossless PNG to prevent compression-induced LSB corruption.
- Tested encoding with 5 standard 512×512 PNG images: Lena, Baboon, Cameraman, Peppers, Boat.

Decoding Activities:

- Implemented `ImageSteganographyService.decode()`: GA seed reproduces the identical position sequence used during encoding.
- LSBs extracted from GA-selected blue channel pixels; bytes assembled until `#END` delimiter detected.
- AES-256 decryption applied to extracted ciphertext to recover original plaintext.
- Tested decoding across all message lengths (100, 500, 1000 characters): 100% recovery accuracy confirmed.

Issue Encountered: JPEG source files caused pixel format mismatch during BufferedImage conversion.

Corrective Measure: Normalized all input pixel formats to TYPE_INT_RGB before embedding to handle JPEG sources correctly.

Phase 9: Audio Steganography — Encode and Decode

Goal: Implement the LSB audio steganography encoding and decoding service for WAV files.

Encoding Activities:

- Implemented AudioSteganographyService.encode(): Reads raw WAV byte array using Java file I/O.
- GA-optimized position sequence selects audio sample bytes for LSB embedding.
- AES-encrypted message bits embedded into the LSB of each selected sample byte.
- ##AUDIO## binary marker appended after ciphertext bits to delimit payload boundary.
- Tested with 5 audio files (30-second WAV, 44.1 kHz): no audible distortion observed in VLC and Windows Media Player.

Decoding Activities:

- Implemented AudioSteganographyService.decode(): GA seed reproduces position sequence; LSBs extracted until ##AUDIO## marker located.
- AES decryption applied; 100% data recovery accuracy confirmed across all audio test cases.

Issue: Ensured ##AUDIO## marker bit pattern does not conflict with embedded ciphertext bits by using a unique 7-byte binary pattern.

Phase 10: Video Steganography — Encode and Decode

Goal: Implement the LSB video steganography encoding and decoding service for MP4 files.

Encoding Activities:

- Implemented VideoSteganographyService.encode(): Processes raw MP4 frame byte data at GA-selected positions within the video byte stream.
- AES-encrypted message bits embedded into frame bytes; ##VIDEO## binary marker appended.
- Tested with 5 ten-second 720p MP4 files: visual and audio quality unaffected; container integrity verified using ffprobe.

Decoding Activities:

- Implemented VideoSteganographyService.decode(): GA seed reproduces position sequence; LSBs extracted until ##VIDEO## marker.
- AES decryption recovers original plaintext; 100% data recovery confirmed.

Issue Encountered: MP4 container format complexity required frame-level byte access without full decode/re-encode cycle.

Corrective Measure: Processed raw byte stream directly to avoid costly full frame-by-frame decode and re-encode, ensuring fast processing.

Phase 11: REST API Development (Week 30)

Goal: Expose all steganography operations as REST API endpoints.

Activities:

- Created SteganographyController with 6 REST endpoints: POST /api/encode/image, /api/encode/audio, /api/encode/video, POST /api/decode/image, /api/decode/audio, /api/decode/video.
- Each encode endpoint accepts: MultipartFile (cover media), String aesKey, int gaSeed, String message. Returns stego-file as downloadable HTTP response (Content-Disposition: attachment).
- Each decode endpoint accepts: MultipartFile (stego-file), String aesKey, int gaSeed. Returns recovered plaintext as JSON.
- Configured CORS in SecurityConfig to allow cross-origin Fetch API requests from the frontend origin.

Issue Encountered: CORS policy blocked Fetch API requests from frontend during local testing.

Corrective Measure: Added @CrossOrigin configuration and CORS filter in SecurityConfig to allow requests from the frontend origin.

Phase 12: Admin Governance Panel

Goal: Build the Admin governance panel and AdminRestController.

Activities:

- Created AdminRestController with governance endpoints: user listing (GET /admin/users), activity history (GET /admin/activity-history), log management, toggle-active (POST /admin/toggle-active), promote-to-admin (POST /admin/promote-admin), platform statistics (GET /admin/stats).
- All /admin/** endpoints restricted exclusively to ROLE_ADMIN via Spring Security filter chain.
- Admin account created by directly inserting into MySQL with role=ROLE_ADMIN, BCrypt password, verified=true, active=true.
- Verified admin credentials are never exposed through the public registration interface.
- Tested all admin endpoints with both ROLE_USER and ROLE_ADMIN tokens to confirm RBAC enforcement.

Phase 13: Upload History Audit Trail

Goal: Implement the upload_history audit trail for all encode/decode operations.

Activities:

- Implemented UploadHistory JPA entity with fields: username, media_type, action_type, filename, timestamp.
- Every successful encode/decode operation creates a new UploadHistory record via UploadHistoryRepository.
- User Activity Log page retrieves and displays the logged-in user's personal operation history.
- Admin Dashboard displays the complete audit trail across all users.

Issue: Fixed timestamp formatting and optimized audit log query for large history tables using pagination.

Phase 14: Frontend Development

Goal: Build the complete responsive frontend web application.

Activities:

- Built responsive web application using HTML5, CSS3, and JavaScript ES6+ with Google Fonts and Font Awesome 6.5.
- Implemented Encoding Hub with sub-panels for Image, Audio, and Video encoding — each with file upload, message input, AES key field, GA seed field, and Encode button.
- Implemented Decoding Hub with sub-panels for each media type — each with stego-file upload, AES key field, GA seed field, Decode button, and result display area.
- Created galaxy-themed dashboard with live HTML5 Canvas starfield animation (animated star particles), glassmorphism sidebar navigation with active state highlighting.
- Activity Log section displays the user's personal operation history retrieved from the MySQL `upload_history` table via REST API.

Issue Encountered: Canvas animation caused performance drops on lower-end browsers due to excessive particle rendering.

Corrective Measure: Optimized canvas frame rate using `requestAnimationFrame` with FPS throttling and reduced particle count for smooth animation across all browsers.

Phase 15: Frontend-Backend Integration and Share Features

Goal: Integrate the frontend with the Spring Boot backend and implement share functionality.

Activities:

- Implemented Fetch API calls in JavaScript to upload media files to Spring Boot REST endpoints using `multipart/form-data`.
- Stego-file returned as downloadable HTTP response and presented to user in browser via a dynamically created anchor element with download attribute.
- Tested complete encode-decode workflow for all three media types end-to-end through browser.
- Implemented share options: download the encoded stego-file, share via WhatsApp (using WhatsApp API link), Gmail (using `mailto` link with attachment), Outlook.

Issue: Fixed minor alignment and UI layout issues discovered during end-to-end browser testing on Chrome, Firefox, and Edge.

Phase 16: Performance Evaluation and Quality Testing

Goal: Evaluate the system's steganographic quality using standard metrics.

Image Evaluation (Week 35):

- Tested with 5 standard 512×512 PNG images: Lena, Baboon, Cameraman, Peppers, Boat.
- Messages of 100, 500, and 1000 characters tested; all AES-256 encrypted before embedding.
- GA configured: population 50, crossover rate 0.8, mutation rate 0.01, max 100 generations.
- Average PSNR: 51.8 dB (GA-optimized) vs 47.3 dB (sequential LSB) — 4.5 dB improvement confirmed.

- MSE below 0.005 and SSIM above 0.9997 in all cases.

Audio and Video Evaluation (Week 36):

- Tested 5 thirty-second WAV files at 44.1 kHz: no audible distortion detected in VLC and Windows Media Player.
- Tested 5 ten-second 720p MP4 files: all stego-MP4 files played correctly with full visual and audio quality.
- Container integrity verified using ffprobe for all stego-MP4 files: declared stream boundaries intact.
- 100% data recovery accuracy confirmed across all media types and all message lengths.

Phase 17: Full System Integration Testing

Goal: Perform complete end-to-end system integration testing.

Activities:

- Tested complete user registration flow: sign up → OTP verification → login.
- Tested encode-decode for Image, Audio, and Video through browser from end to end.
- Verified Admin panel: user management, activity log, toggle-active, promote-admin.
- Validated audit trail entries created correctly for every encode/decode operation.

Issue: Debugged minor integration issues in admin panel statistics refresh after operations (stale data shown).

Corrective Measure: Added explicit page data refresh call after each admin operation to ensure up-to-date statistics.

Phase 18: Guide Review and Refinement

Goal: Present the complete project to the guide and apply suggested improvements.

Activities:

- Presented complete project execution and workflow to Prof. Yogita More.
- Demonstrated all three media type encoding/decoding workflows live in browser.
- Applied guide's suggested UI improvements: improved button labels, added success toast notifications, refined error messages.
- Implemented all guide suggestions and verified changes did not affect core steganography functionality.

Phase 19: Final Testing

Goal: Ensure error-free execution across all modules before submission.

Activities:

- Performed final end-to-end testing of the complete StegnoVault system.
- Tested all 6 REST endpoints with valid and invalid inputs (wrong file format, missing parameters, incorrect AES key, incorrect GA seed).
- Verified error-free execution for all encode/decode operations across Image, Audio, and Video.

Bug Fixed: Fixed final edge-case bug in audio `##AUDIO##` marker detection for very short messages (< 10 characters) where the marker was detected prematurely.

Corrective Measure: Adjusted marker detection logic to require a minimum payload size before beginning marker scan.

Phase 20: Final Documentation and Research Paper

Goal: Complete all project documentation and prepare for final evaluation.

Activities:

- Completed research paper: StegnoVault — A Web-Based Secure Data Transmission System Using LSB Steganography (published-format IEEE paper with abstract, system architecture, methodology, experimental results, and references).
- Documented all modules: AES-256 encryption, Genetic Algorithm optimizer, LSB steganography for Image, Audio, and Video.
- Prepared final project report covering architecture, methodology, experimental results, and conclusion.
- Completed project presentation slides and submitted project for final evaluation.
- Verified final formatting and content accuracy across all report sections before submission.

6.2 Working Model of Proposed System

The following describes the complete working model of StegnoVault as it executes from the user's perspective, from initial registration through to successful hidden message transmission and recovery.

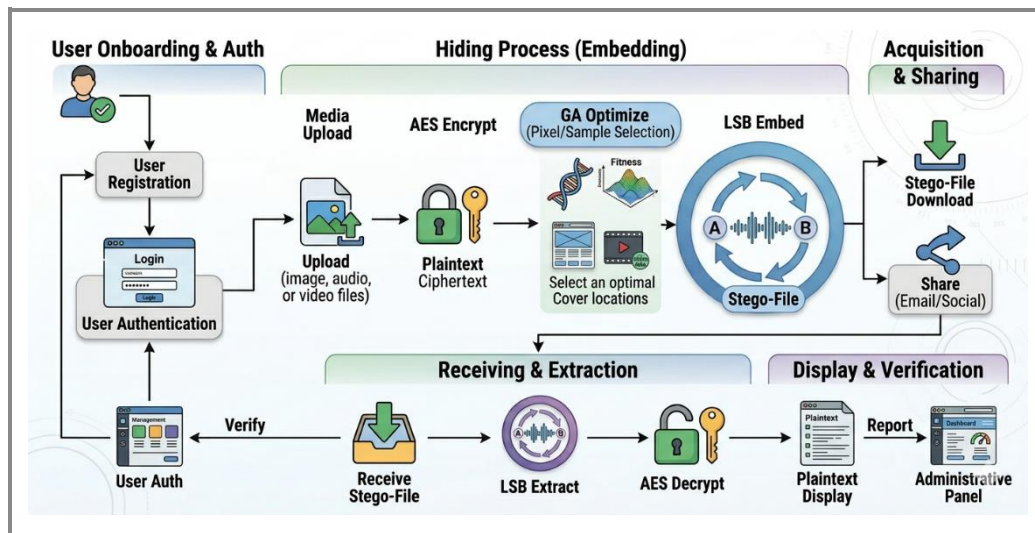


Fig. 6.2: Working Model of StegnoVault — Complete System Flow

Step 1: User Registration and Authentication

A new user opens the StegnoVault registration page in any modern web browser. The user enters their full name, email address, mobile number, password, and confirm password. Upon submission, the system validates the inputs (email format, password match, field completeness), checks for duplicate email, hashes

the password using BCrypt (strength 12), creates a User record (verified=false), generates a 6-digit OTP via SecureRandom, and sends it to the user's Gmail inbox via Spring Mail/SMTP. The user enters the OTP, the system validates it (must match and not be expired within 10 minutes), and sets verified=true. The user is now able to log in.

Step 2: Login and Dashboard Access

The authenticated user logs in with email and password. Spring Security validates credentials via BCrypt comparison. On success, an HTTP session is established and the user is redirected to the galaxy-themed dashboard. The dashboard displays: the HTML5 Canvas starfield animation, the glassmorphism sidebar with links to Encoding Hub, Decoding Hub, Activity Log, and Profile, and the user's recent operation history from upload_history.

Step 3: Selecting and Uploading Cover Media

The user navigates to the Encoding Hub and selects the appropriate tab (Image / Audio / Video). The user uploads a cover media file using the file upload control. The system accepts PNG/JPEG for images, WAV for audio, and MP4 for video. A preview or file name confirmation is displayed after successful upload.

Step 4: AES Encryption of Secret Message

The user types the secret message in the text area, enters an AES-256 key (any string), and provides a GA seed (any integer). The user must communicate the AES key and GA seed to the intended receiver through a separate secure channel (e.g., in-person, encrypted messaging). Upon clicking Encode, the AESCipherService encrypts the message using AES/CBC/PKCS5Padding with a 256-bit key, producing a ciphertext binary bit string.

Step 5: Genetic Algorithm Position Optimization

The GeneticOptimizerService is invoked with the provided GA seed and the media size. The GA runs for up to 100 generations, evaluating 50 candidate position sequences per generation based on entropy score and spatial distribution score. The highest-fitness sequence is returned as the int[] embedding map.

Step 6: LSB Embedding and Stego-File Generation

The media-specific steganography service embeds the ciphertext binary bit string into the cover media at the GA-selected positions, modifying only the LSB of each selected pixel (blue channel) or sample byte. A delimiter is appended after the last bit. The resulting stego-file is returned as an HTTP response with Content-Disposition: attachment. The browser automatically prompts the user to download the stego-file.

Step 7: Audit Logging

A UploadHistory record (ENCODE) is automatically created and saved to the MySQL upload_history table with the username, media type, action type, filename, and timestamp. This entry appears in the user's Activity Log and in the Admin Dashboard's complete audit trail.

Step 8: Stego-File Sharing

The user shares the stego-file with the receiver using the built-in share options: direct download (the receiver is sent the file via any file transfer method), or direct share via WhatsApp, Gmail, or Outlook. The stego-file looks and plays exactly like the original — no visible or audible difference can be detected.

Step 9: Decoding — LSB Extraction

The receiver logs in to StegnoVault, navigates to the Decoding Hub, selects the correct media type tab, uploads the stego-file, enters the matching AES key and GA seed, and clicks Decode. The GeneticOptimizerService reproduces the identical position sequence using the same GA seed. The media-specific service reads the LSB of each GA-selected position in order, assembling bytes until the delimiter is detected.

Step 10: AES Decryption and Message Display

The AESCipherService decrypts the extracted ciphertext using the provided AES key through inverse AES operations (InvShiftRows, InvSubBytes, InvMixColumns, AddRoundKey). If the correct AES key is provided, the original plaintext message is perfectly recovered and displayed in the Decoding Hub text area. If an incorrect key is provided, decryption fails with a padding error and an appropriate error message is shown. A UploadHistory record (DECODE) is saved for the operation.

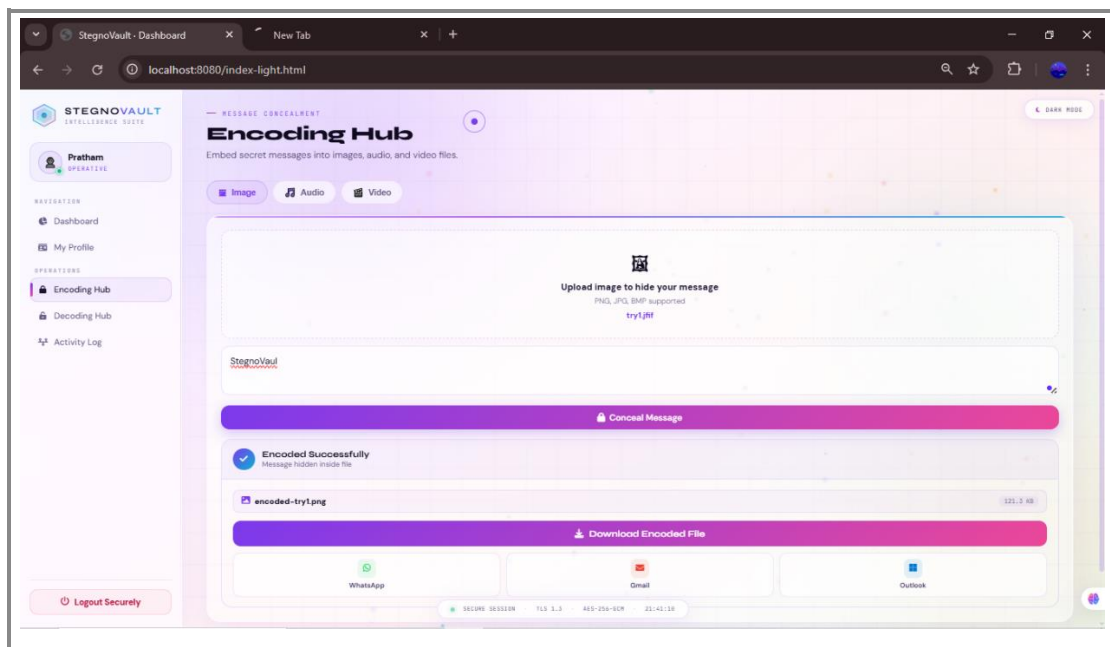


Fig. 6.3: Encoding Hub — Web Interface Screenshot

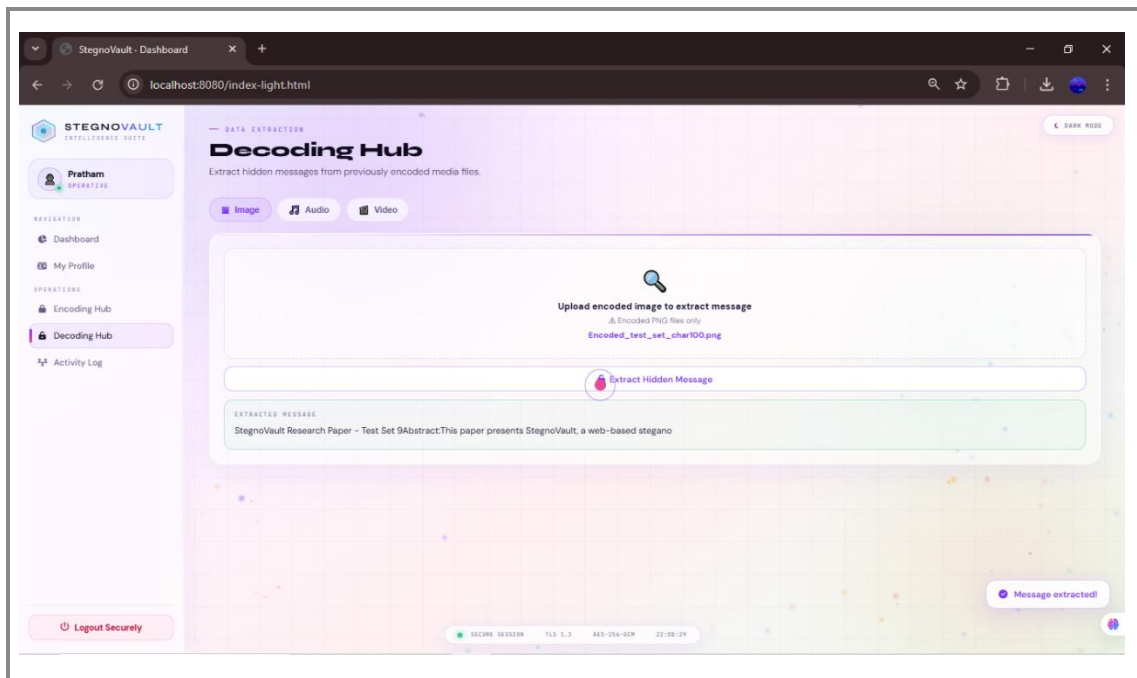


Fig. 6.4: Decoding Hub — Web Interface Screenshot

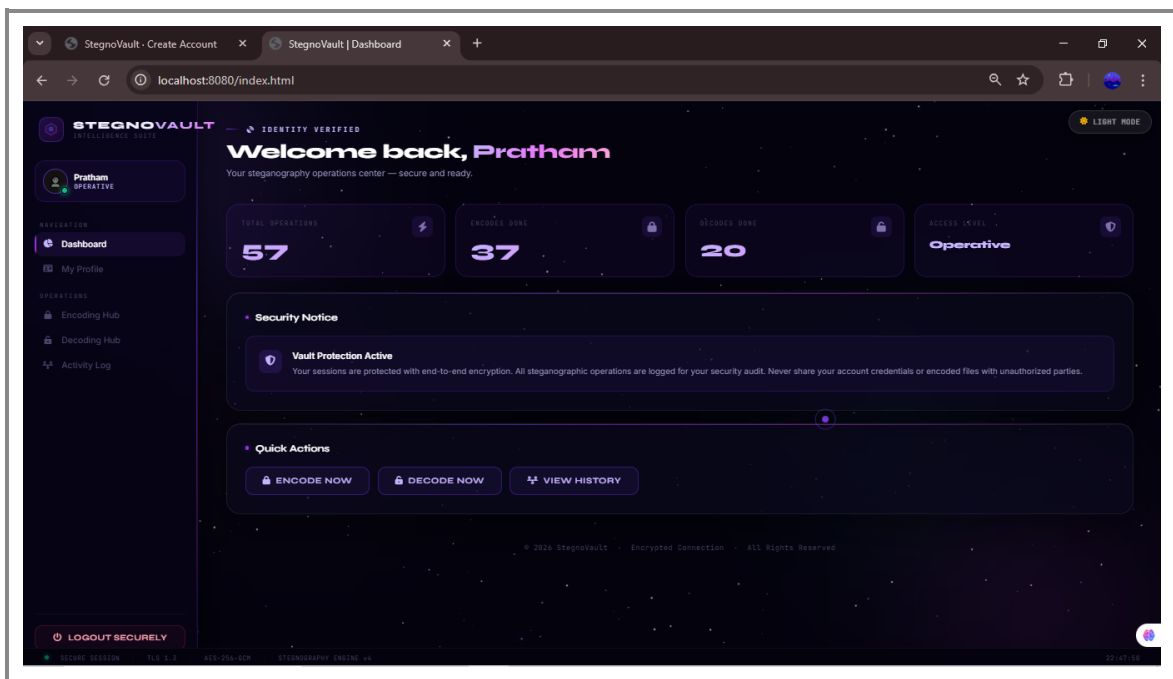


Fig. 6.5: Galaxy-Themed Dashboard — Screenshot

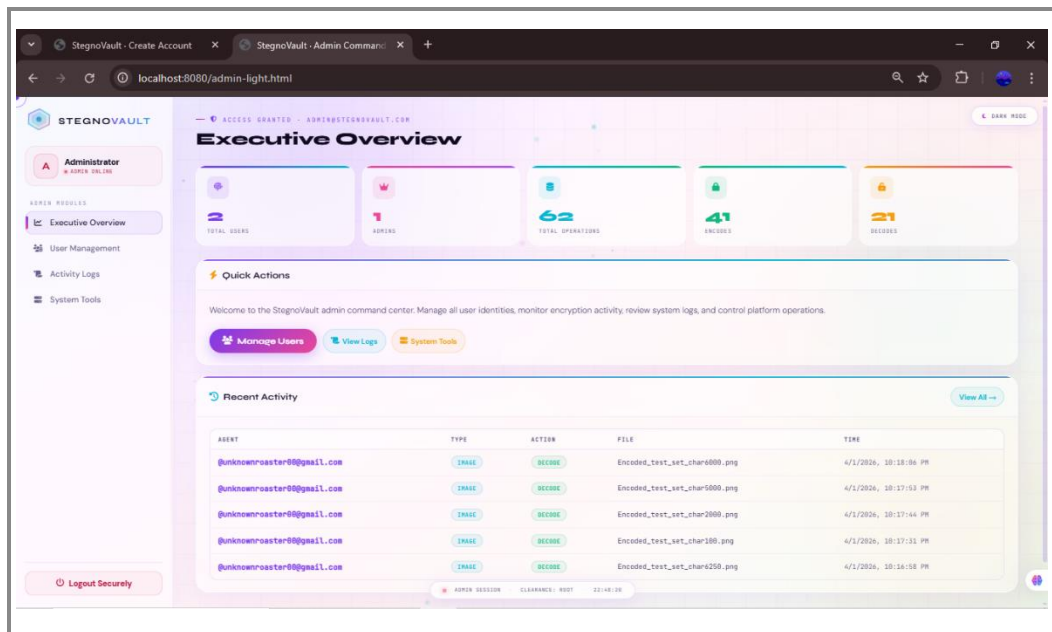


Fig. 6.6: Admin Panel — Screenshot